

Pipeline-Orchestrator — workflows usuais (legenda das cores)

Cada página é um workflow rodado na sua máquina (os fluxos Paperclip/servidor ficam de fora). A cor indica quem executa cada passo.

Passo inline do orquestrador

Subagente auxiliar (especialista)

Gate — você decide / aprova

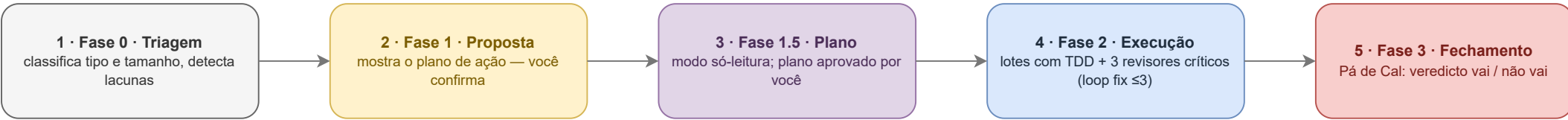
Modo plano (só-leitura)

Crítico / Pá de Cal (veredito)

Fechamento / entrega

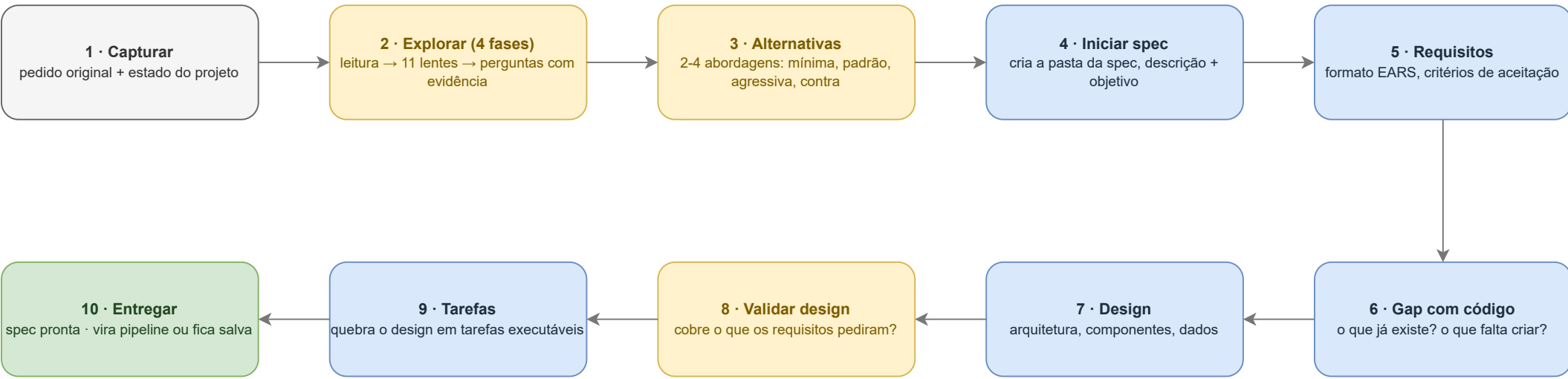
1 · Pipeline mestre — as quatro fases

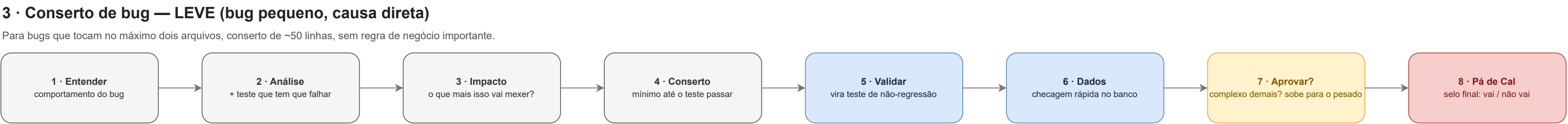
O coração do plugin. Toda tarefa passa por triagem, proposta, plano, execução em lotes com TDD e revisão crítica, e fecha com o veredito Pá de Cal.



2 · Brainstorm — preparação antes do pipeline

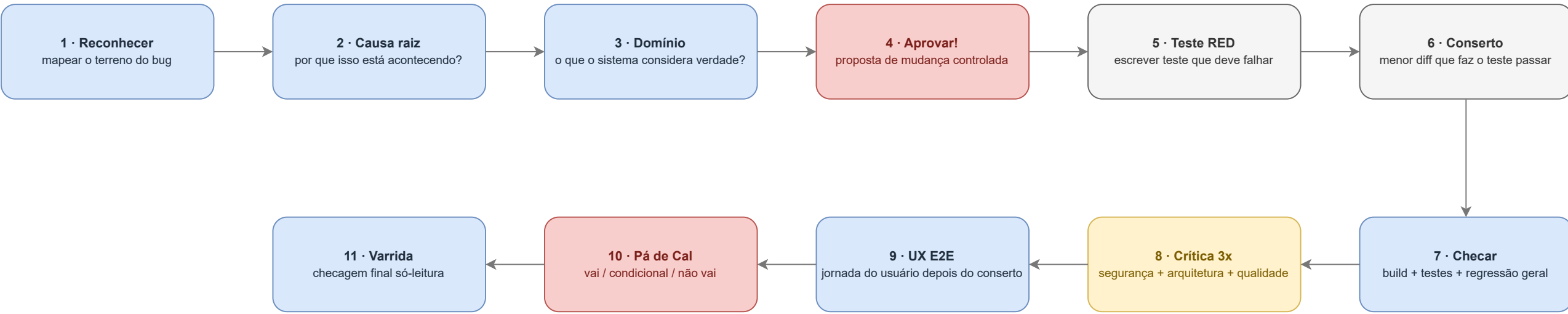
Roda antes de construir, quando ainda falta esclarecer o que você quer. Produz uma especificação completa pronta para virar pipeline.



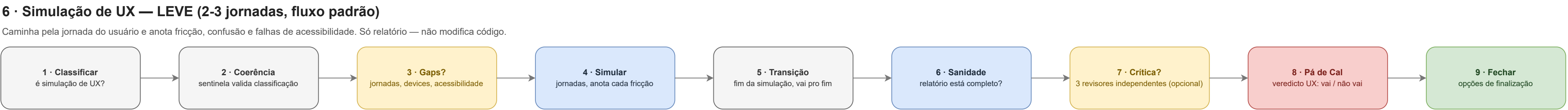


4 · Conserto de bug — PESADO (bug complexo / incidente em produção)

Para bugs com dados, concorrência, vários módulos ou impacto em produção. Aprovação obrigatória da proposta antes de mudar qualquer linha.

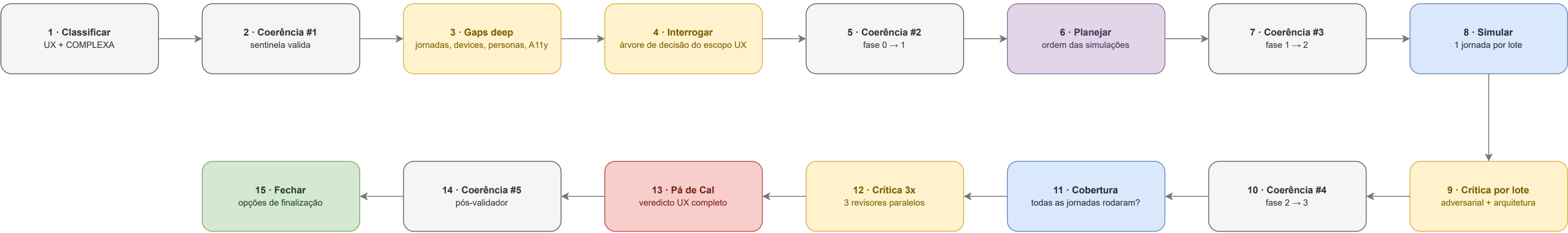






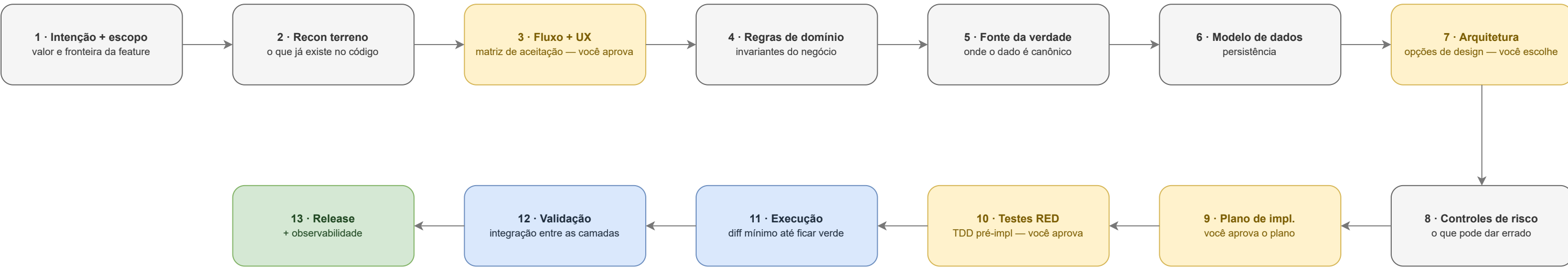
7 · Simulação de UX — PESADO (auditoria E2E, várias personas, cross-device)

Acrescenta interrogação de design, planejamento da ordem das simulações e revisão crítica por lote dentro da execução.



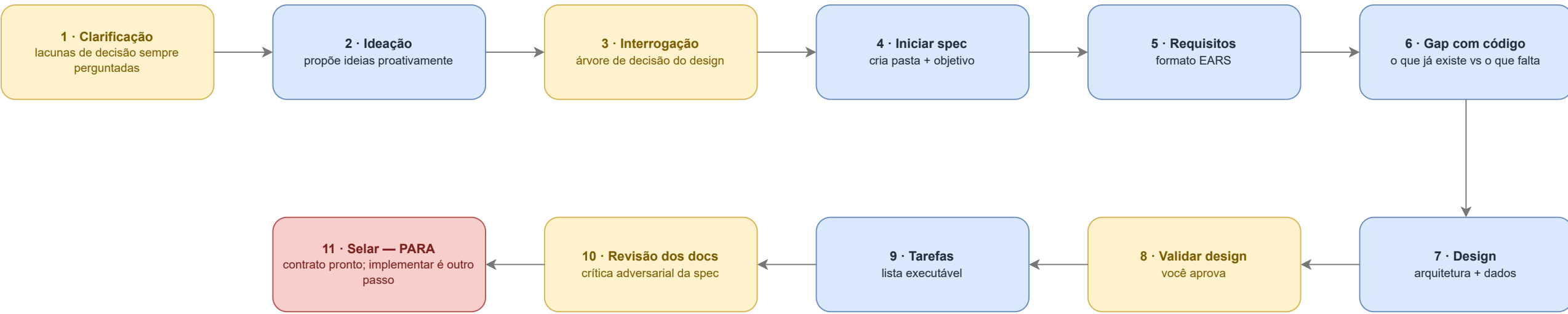
8 · Funcionalidade nova / User Story (13 passos · modo pesado)

Constrói uma feature com previsibilidade máxima. User Story usa a mesma sequência, extraindo antes os critérios de aceitação da frase ("Como X, quero Y"). 4 gates obrigatórios.



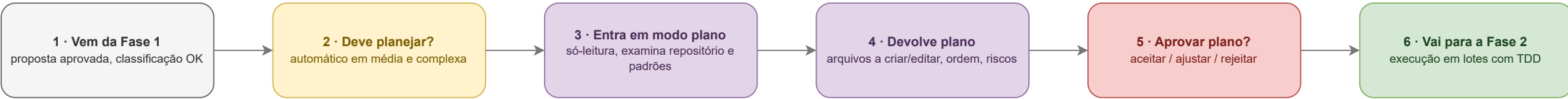
9 · Spec — autoria (cria a especificação e PARA, sem implementar)

Gera uma especificação grau-contrato do zero e a sela. Implementar é um passo separado depois.



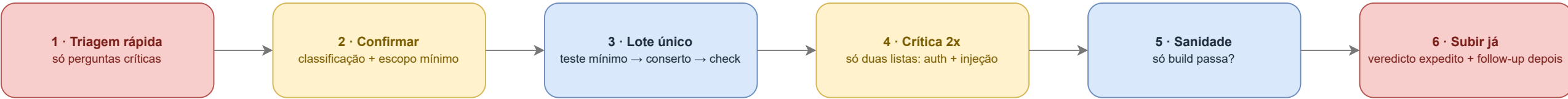
10 · Plano (modo de planejamento) — fase dentro do pipeline

Não é um pipeline próprio: entra antes da execução. Automático em tarefas média e complexa; opcional nas simples. Modo só-leitura, você aprova antes de qualquer mudança.



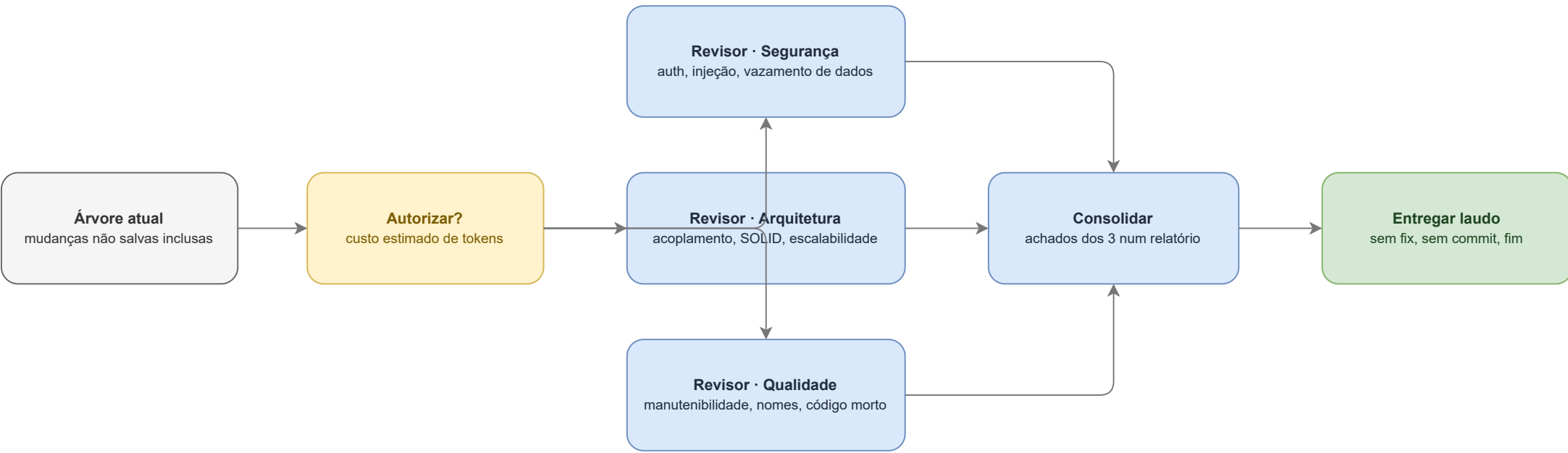
11 · Hotfix (emergência) — reduz escopo, não segurança

Para quando o sistema está parado e você precisa subir um conserto agora. Corta cerimônia, mantém o conserto, a revisão de segurança e o veredito.



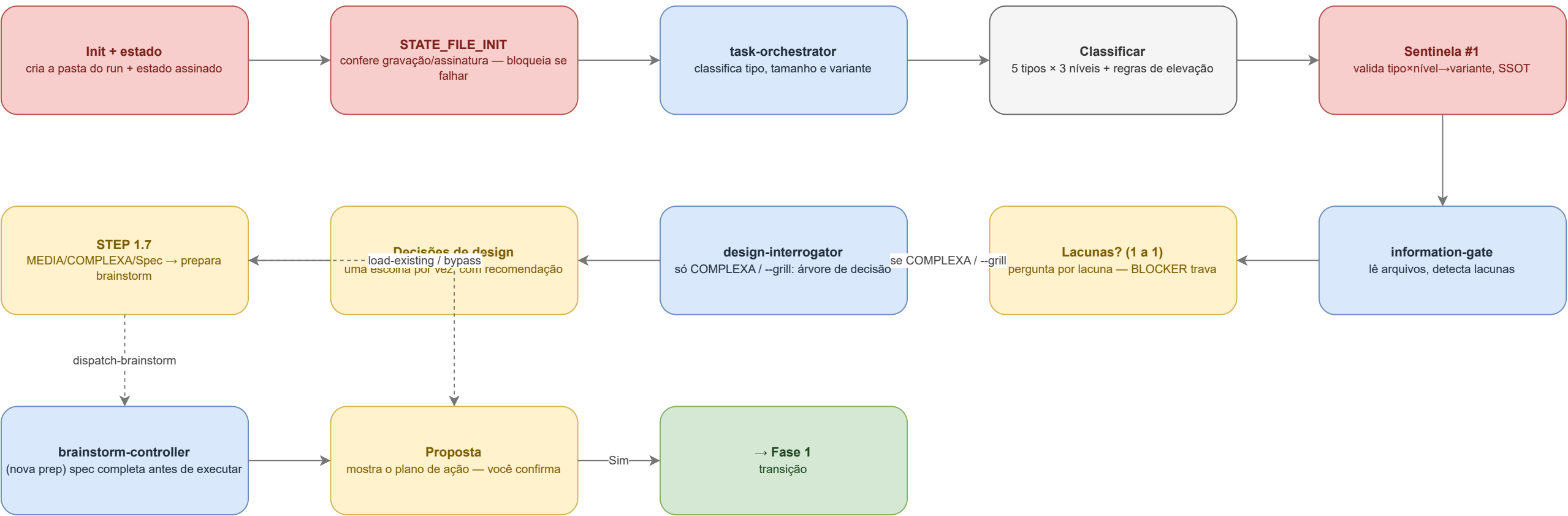
12 · Revisão isolada (review-only)

Roda só a equipe crítica final sobre o código que você já escreveu. Três revisores em paralelo, zero contexto compartilhado. Só relatório — nada é consertado nem commitado.



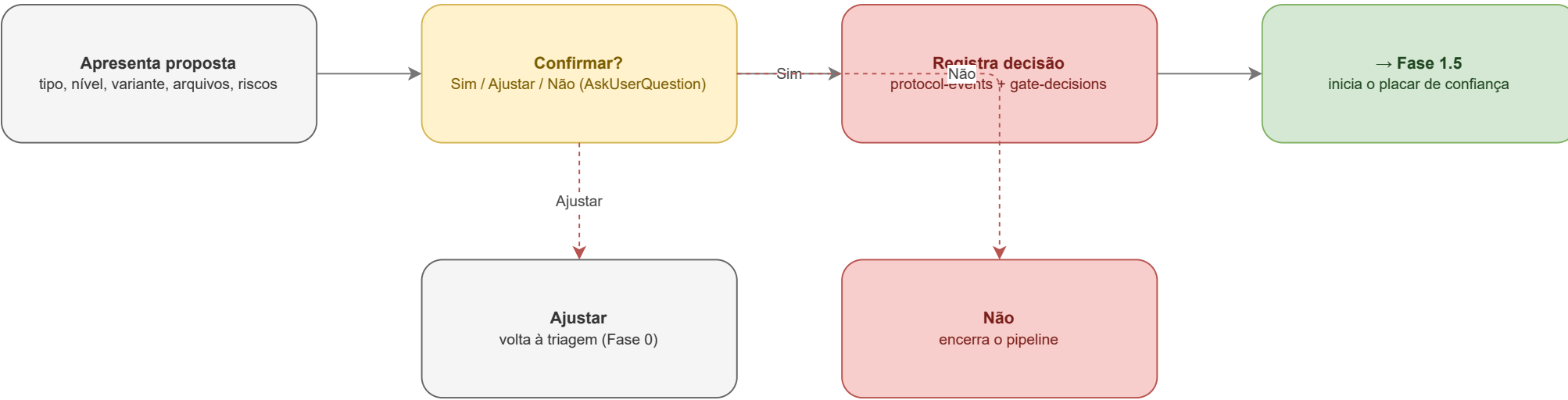
Fase 0 · Triagem — detalhe

Classifica a tarefa, fecha lacunas de informação e decide o roteamento antes de propor. Cada subagente fala com você pelo protocolo de handshake (ver página própria).



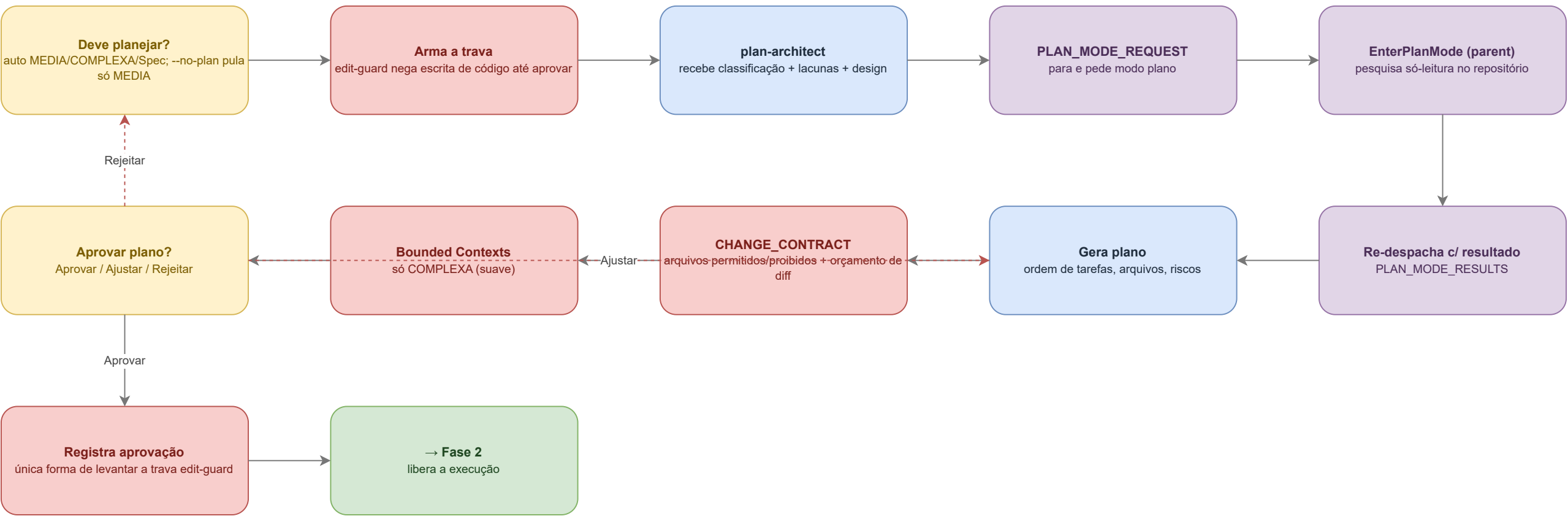
Fase 1 · Proposta — detalhe

Apresenta a proposta e exige sua confirmação antes de planejar. Você pode aprovar, mandar ajustar (volta à triagem) ou recusar (encerra).



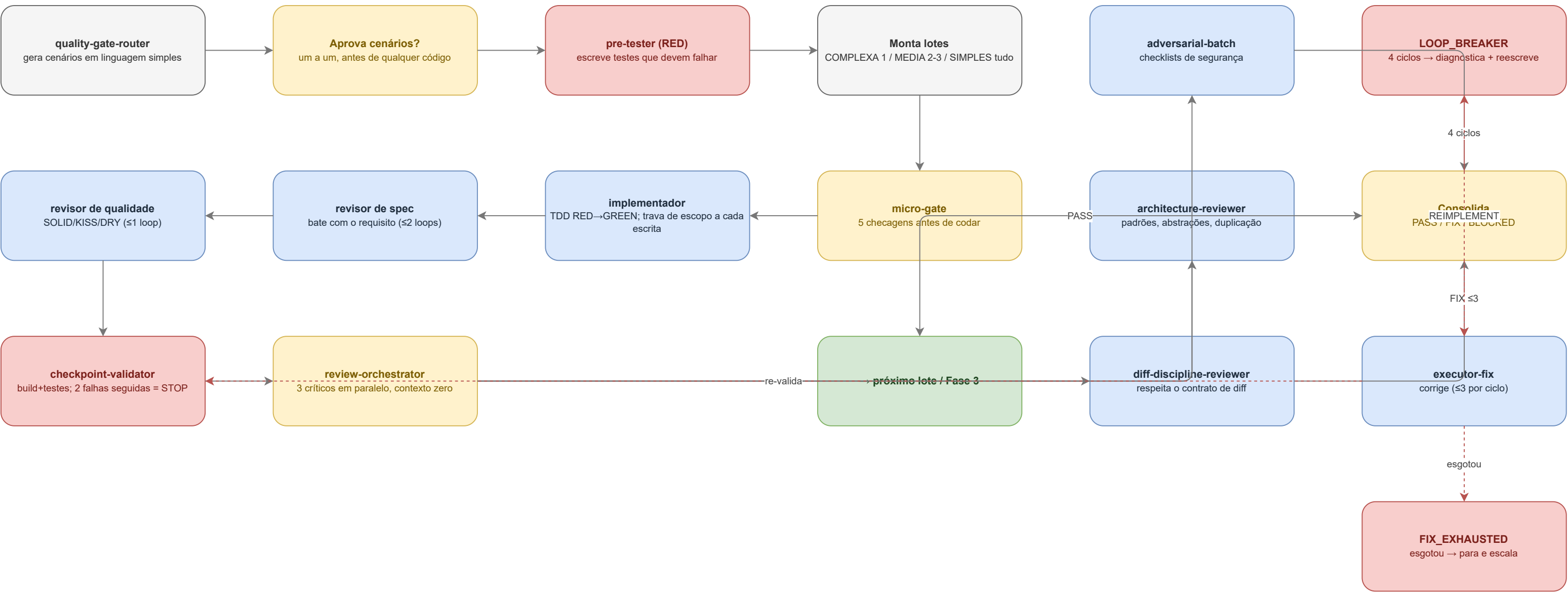
Fase 1.5 · Plano (modo de planejamento) — detalhe

Automático em MEDIA/COMPLEXA/Spec, opcional via --plan. Uma trava determinística impede escrever código até o plano ser aprovado por você.



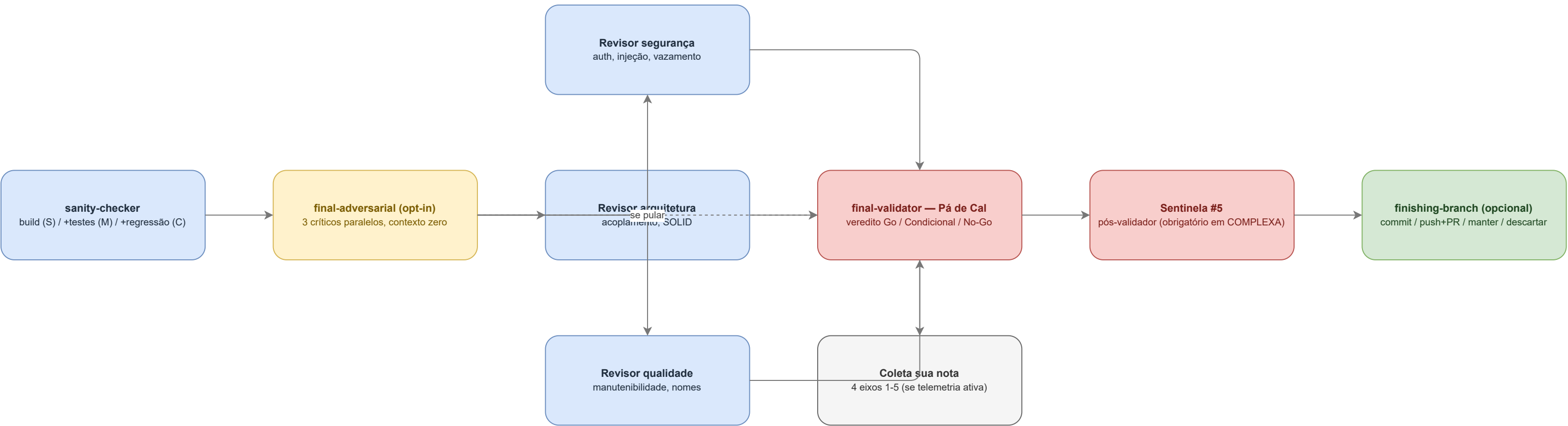
Fase 2 · Execução em lotes — detalhe

A fase mais densa. Primeiro os testes que devem falhar (TDD RED), depois lote a lote: cadeia por tarefa, checkpoint, três críticos em paralelo (contexto zero) e o loop de correção com disjuntor.



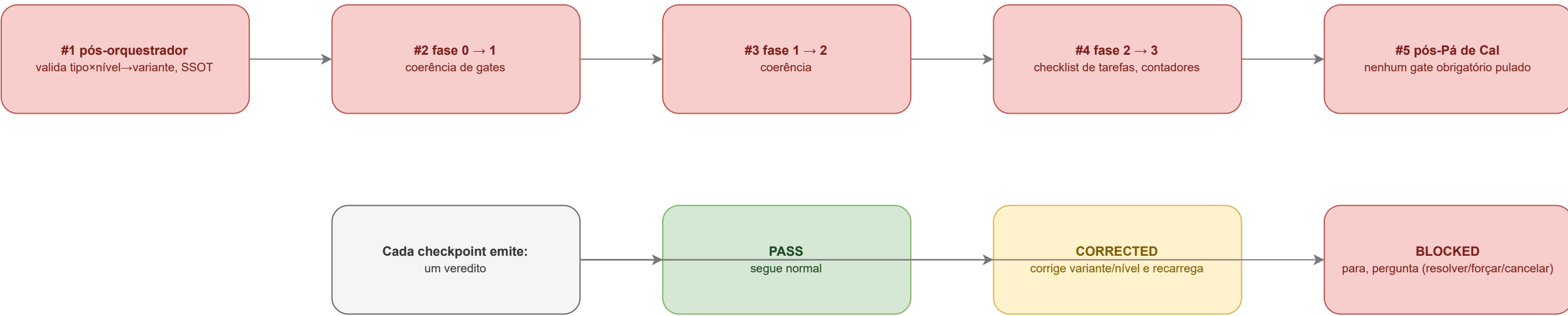
Fase 3 · Fechamento — detalhe

Sanidade proporcional, time adversarial final opcional (custa tokens), o veredito Pá de Cal, e a finalização do branch se for o caso.



Sentinela — 5 checkpoints obrigatórios

Roda em pontos fixos de todas as fases, com contexto isolado da implementação. Cada checkpoint emite um de três veredictos.



Protocolo de handshake (Achado #7) — como os subagentes falam com você

Subagentes não têm acesso direto a perguntar/despachar/planejar. Eles emitem um bloco, param, e o orquestrador-pai resolve e re-despacha — repetindo até o resultado final.

